

USED CAR PRICE

ระบบทำนายราคารถมือสอง

ปัญหา

ปัญหาของผู้ใช้

ผู้ขายรถมือสองรายย่อยไม่รู้ว่าควรตั้งราคาเท่าไร

- ตั้งต่ำไป → ขาดทุน
- ตั้งสูงไป → ขายไม่ออก ประกาศค้างนาน

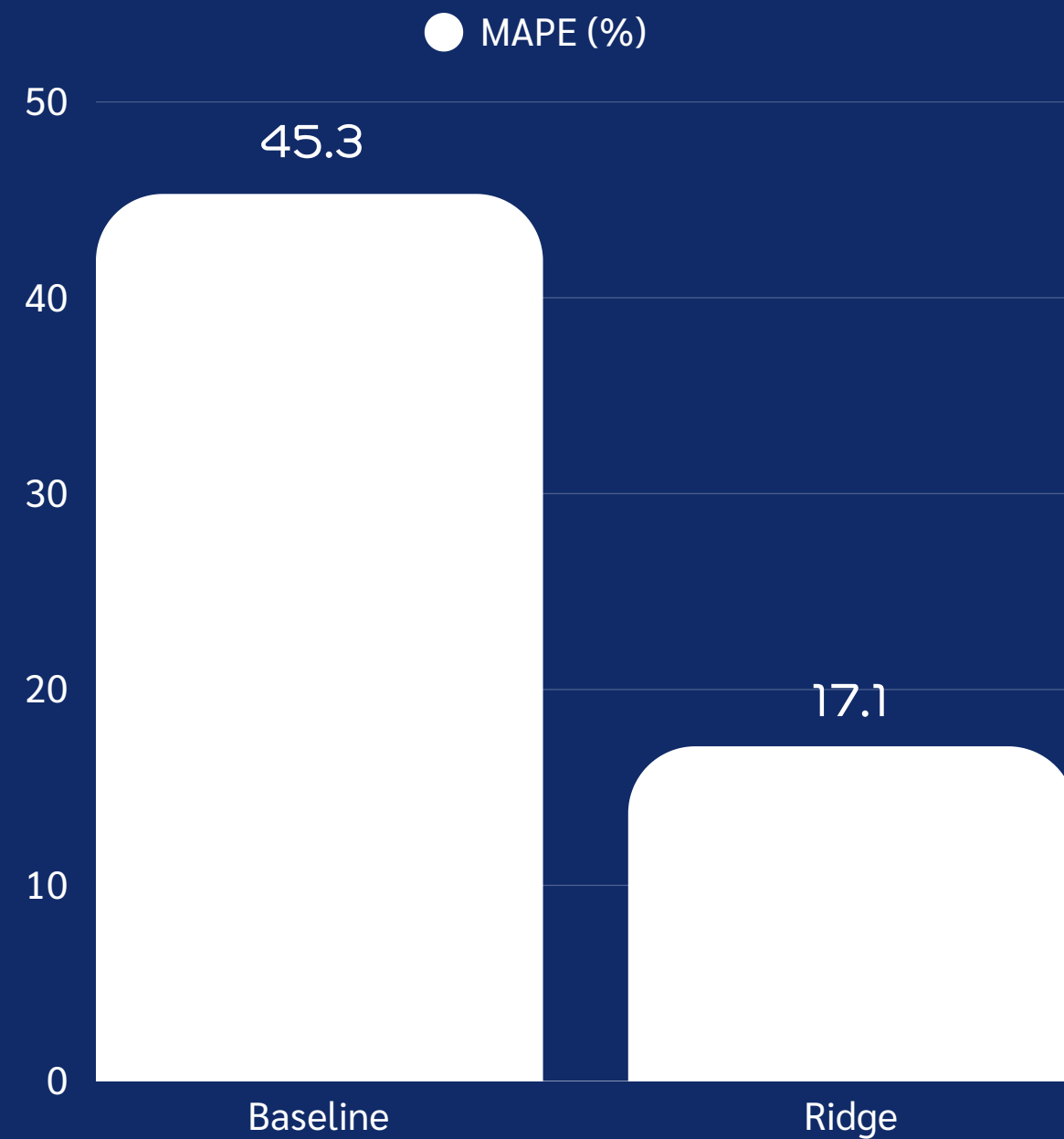
ต้องไล่เทียบประกาศตัวเอง ใช้เวลามาก
และไม่มีเกณฑ์ชัดเจนว่าราคาไหนเหมาะสม



ทำไมตั้งราคาด้วยตัวเองไม่ได้

- ราคาขึ้นกับหลายปัจจัยพร้อมกัน
ปี เลขไมล์ ยี่ห้อ เชื้อเพลิง เกียร์ เจ้าของ กำลังเครื่อง
- $32 \text{ ยี่ห้อ} \times \text{เชื้อเพลิง} \times \text{เกียร์} \times \text{เจ้าของ}$
= สูงสุด 1,280 กลุ่ม (ยังไม่นับปีและเลขไมล์)
- ตลาดเปลี่ยนตามเวลา
ข้อมูลรถรุ่นใหม่ต่างจากรุ่นเก่าใน 11 จาก 12 คอลัมน์
ต้องแก้กฎซ้ำทุกครั้ง

ทำไมต้องใช้ ML



- BASELINE (ทนายราคากลางตายตัว): MAPE 45.3%
- RIDGE REGRESSION (ML): MAPE 17.1% (วัดบนชุด VALIDATION)
- แม่นยำขึ้นกว่า 2.6 เท่า
- ข้อมูลมี 32 ยี่ห้อ x เชื้อเพลิง x เกียร์ x เจ้าของ เขียนกฎเองไม่ไหว

MAPE = ความคลาดเคลื่อนเฉลี่ยเป็น % (ยิ่งต่ำยิ่งดี)

AI Project Canvas

1. Value Proposition (คุณค่าที่ระบบสร้าง)

- ผู้ขายรถมือสองรายย่อย
- ลดความเสี่ยง 2 ด้าน
- platform ประกาศขายรถ

2. Customers/Users (กลุ่มผู้ใช้งาน)

- ผู้ขายรายบุคคล
- Dealer หรือ Trust Dealer
- ผู้ซื้อ

3. Stakeholders (ผู้มีส่วนได้ส่วนเสีย)

- เจ้าของ platform หรือ ทีมธุรกิจ
- ทีม ML/MLOps
- ทีมให้บริการลูกค้า
- ผู้สอบ หรือ ผู้ตรวจ

4. ML Task และเหตุที่ใช้ ML แทนการเขียนกฎ

งาน:

Regression โดยทำนาย selling_price (รูปี) จาก 12 feature

ทำไมกฎธรรมดาไม่พอ:

- ราคาขึ้นกับหลายปัจจัยพร้อมกัน
- มี 30+ ยี่ห้อ*เชื้อเพลิง*เกียร์*จำนวนเจ้าของ

จำนวนกฎที่ต้องเขียนเอง และต้องแก้ทุกครั้งที่ตลาดเปลี่ยน

- baseline แบบ “ใช้ราคากลาง (median) ของตลาด” ได้ val MAPE 45.3% ขณะที่ Ridge ได้ 17.1%
- ML เทรนใหม่จากข้อมูลล่าสุดได้อัตโนมัติเมื่อตลาดเปลี่ยน

5. Data (ข้อมูลที่ใช้)

- แหล่งข้อมูล
- ขนาดข้อมูล
- การแบ่งชุดข้อมูล (ตามปีรถ)
- การตรวจคุณภาพ
- ข้อจำกัด

6. Output และ Integration (ผลลัพธ์/การใช้งาน)

Output (API Response)

- predicted_price_inr
- predicted_price_thb
- model_version

Integration

- REST API (POST/ predict) แบบ real-time ใน Docker container เรียกจากหน้าลงประกาศได้ทันที

7. Metrics (ตัวชี้วัด)

- Optimizing metric MAE: 177,922 INR
- Gating metric MAOE $\leq$ 20%: 19.2%
- ระบบ (SLO) ต้องผ่านด้วย
 - p95 latency $\leq$ 250 ms
 - error rate $\leq$ 1%

8. เชื่อมตัวชี้วัดโมเดลกับตัวชี้วัดธุรกิจ

8.1 MAE 177,922 รูปี หมายถึงอะไร (test set, 50ปี 2018–2020)

8.2 สัดส่วนทำนายบน test set (รถปี 2018–2020)

8.3 ความแม่นยำแยกตามช่วงราคา (test set)

9. ผู้ได้รับผลกระทบเมื่อโมเดลทำนายผิด

- ผู้ขาย
- ผู้ซื้อ
- Dealer
- platform

10. Data Drift และ Concept Drift

Data Drift

- ตรวจพบ 91.7% ของ column
- ส่วนใหญ่เป็นฟิลเลอร์เต็มตัวเลย

Concept Drift

- MAE Ratio 1.53
- โมเดลปัจจุบันเสื่อมประสิทธิภาพ

11. Cost (ต้นทุน)

- Infrastructure
- Data
- Team Effort
- ค่าใช้จ่ายโดยรวมเป็นเชิงคุณภาพ ไม่ใช่ตัวเลขต้นทุนจริง (เหมาะสำหรับโปรเจกต์การเรียนรู้/ต้นแบบ)

12. Revenue/ Benefit (รายได้/ ประโยชน์ที่คาดหวัง)

- เพื่อโอกาสขายสำเร็จ
- ฟีเจอร์ Premium
- ลดทุนบริการลูกค้า
- ประโยชน์ระยะยาว
- หมายเหตุ

13. คำถามตรวจสอบหัวข้อโครงการ (Checklist)

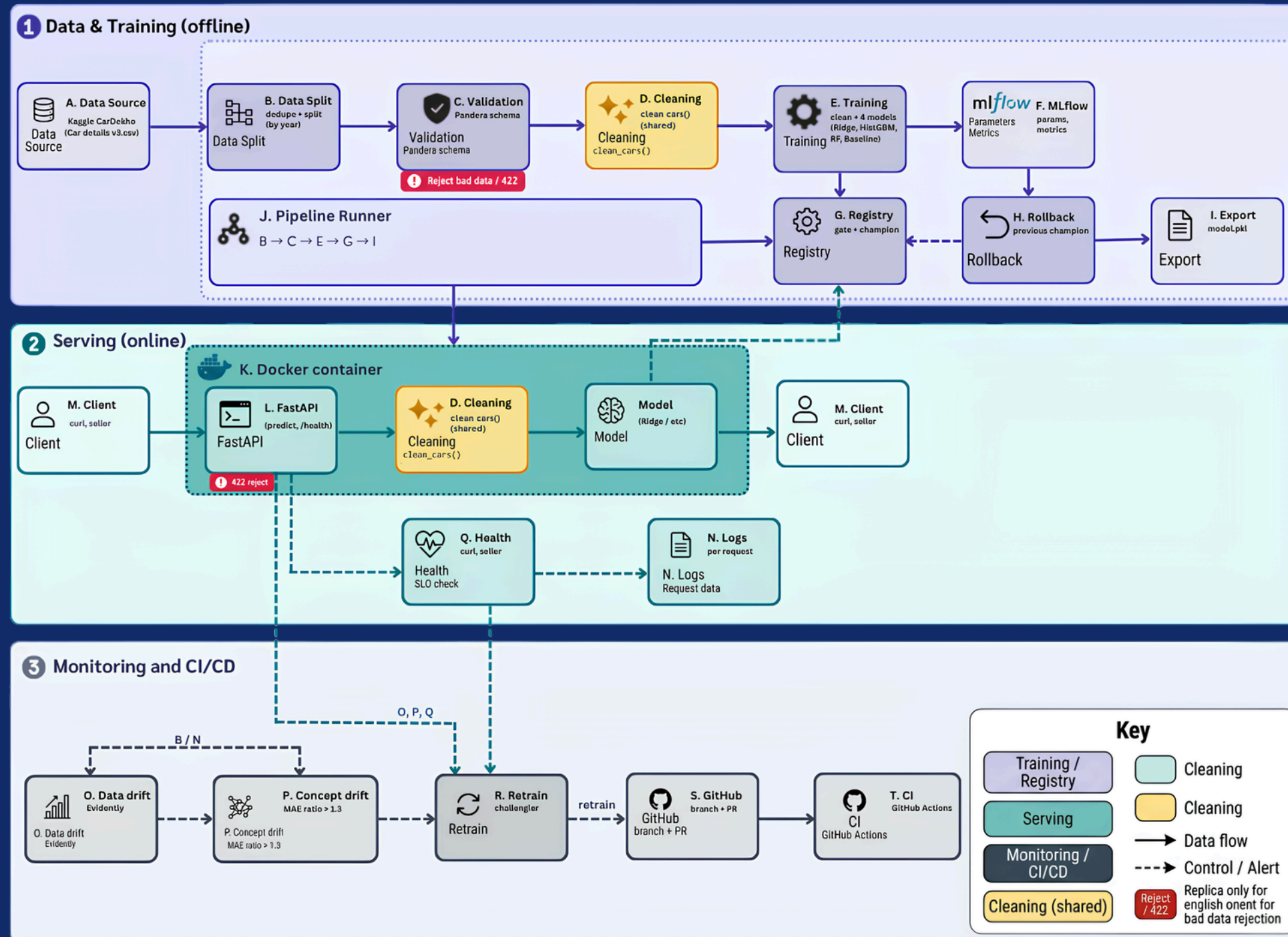
- 1) แก่ปัญหาอะไร และมีคุณค่าอย่างไร (value Proposition)?
- 2) ใครคือผู้ใช้ และผู้มีส่วนร่วมได้ส่วนเสีย?
- 3) หากทำนายผิด ใครได้รับผลกระทบ และจะลดผลกระทบอย่างไร?
- 4) มีแผนติดตามและเทรนใหม่อย่างไร?
- 5) โครงการนี้มีความเป็นไปได้และเหมาะสมกับการนำไปใช้จริงหรือไม่?
- 6) คาดว่าจะเกิดรายได้ หรือ ประโยชน์อย่างไร?
- 7) มี Data Drift/Concept Drift หรือไม่ และจะจัดการยังไง?
- 8) เชื่อมตัวชี้วัดโมเดลกับชี้วัดธุรกิจได้อย่างไร?
- 9) วัดผลด้วยตัวชี้วัดอะไร และผลเป็นอย่างไร?
- 10) มีต้นทุนอะไรบ้าง?

ข้อมูล

- แหล่งข้อมูล: Vehicle Dataset from CarDekho (Kaggle) ไฟล์ Car details v3.csv
- ขนาดข้อมูล: 8,128 แถว ลบข้อมูลซ้ำ 1,202 แถว เหลือ 6,926 แถว
- Target: selling_price (รูปี)
- Feature 12 ตัว: year, km_driven, mileage, engine, max_power, torque, seats, brand, fuel, seller_type, transmission, owner

ชุดข้อมูล	เงื่อนไข	จำนวน
Train	ปี $\leq$ 2016	5,100
Valiadtion	ปี = 2017	808
Test	ปี $\geq$ 2018	1,018

Architecture Diagram



1. Data & Training “สร้างโมเดล”

เตรียมข้อมูล สร้างโมเดล และจัดการเวอร์ชันโมเดล ให้เข้าใจว่า ข้อมูลรถ ได้ผ่านการตรวจสอบและทำความสะอาดจากนั้นให้ Train จนได้โมเดล ก่อนที่ระบบจะทำนายราคาได้ เราต้องสร้างและตรวจสอบโมเดลให้เรียบร้อยก่อน

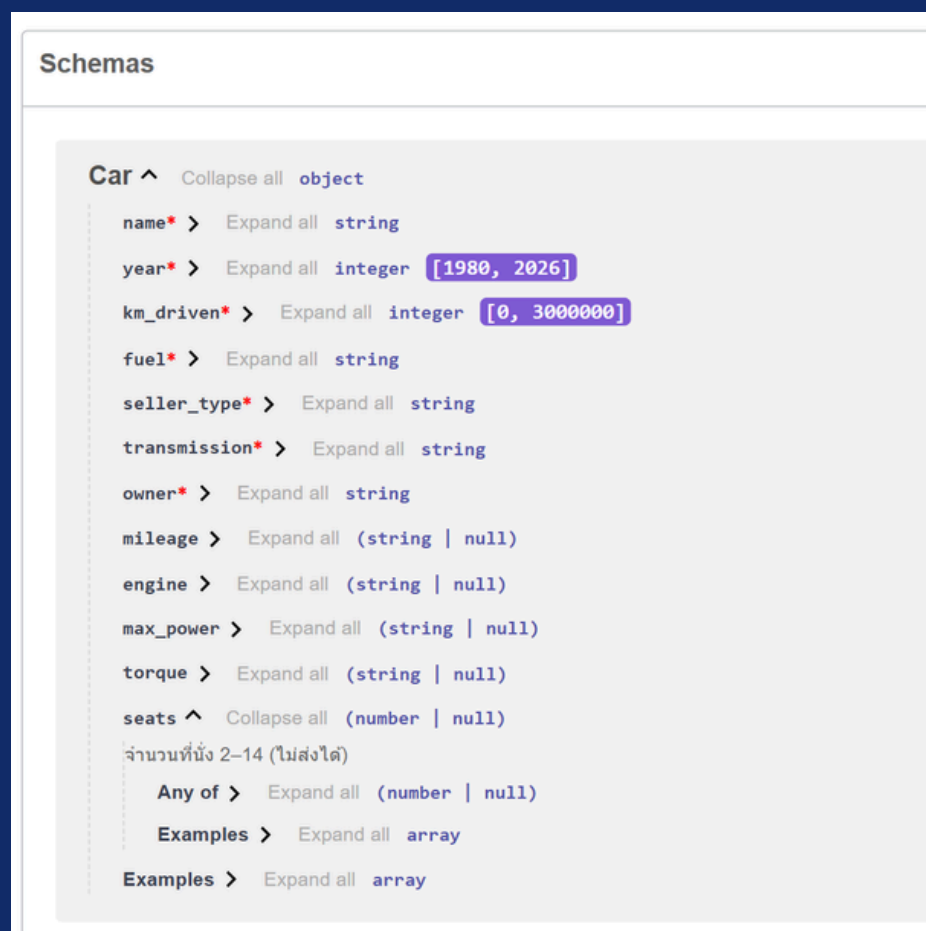
2. Serving “เอาโมเดลไปใช้งานจริง”

ผู้ใช้ส่งข้อมูลรถเข้ามาในระบบผ่าน API ระบบจะทำการ Cleaning และเตรียมข้อมูลก่อนนำไปประมวลผล จากนั้น Model จะนำข้อมูลไปทำนายราคารถมือสอง FastAPI, Cleaning และ Model ทำงานอยู่ภายใน Docker Container เดียวกัน สุดท้ายระบบจะแสดงราคาที่ทำนายให้ผู้ใช้

3. Monitoring & CI/CD “ดูแลโมเดลหลัง Deploy”

ระบบจะตรวจสอบว่าข้อมูลรถใหม่มีการเปลี่ยนแปลงจากข้อมูลที่ใช้ Train หรือไม่ โดยดูทั้ง Data Drift และ Concept Drift ถ้าค่าการเปลี่ยนแปลงสูงเกิน Threshold ระบบจะทำการ Retrain โมเดล เมื่อได้โมเดลใหม่แล้ว จะส่งขึ้น GitHub และเข้าสู่กระบวนการ CI/CD ระบบจะทำการทดสอบโมเดลก่อนนำไป Deploy ใช้งานจริง

การตรวจสอบข้อมูล



```
PS C:\Users\User\UsedCarPrice-mlops> curl.exe -s -w " HTTP %{http_code}\n" -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d "@$env:TEMP\car_seats100.json"
{"detail":[{"type":"less_than_equal","loc":["body","seats"],"msg":"Input should be less than or equal to 14","input":100,"ctx":{"le":14.0}}]} HTTP 422
```

Model and Tuning

					MAE	MAPE
● hist_gbm	✓ 1 day ago	5.3s	-	-	129150.3227...	0.143912674...
● random_forest	✓ 1 day ago	5.1s	-	-	135776.3589...	0.149776062...
● ridge	✓ 1 day ago	21.5s	184905.9682...	0.196171131...	120614.8227...	0.170662199...
● baseline_median	✓ 1 day ago	7.1s	-	-	415936.8490...	0.452976398...
● hist_gbm	✓ 3 days ago	4.6s	-	-	129150.3227...	0.143912674...
● random_forest	✓ 3 days ago	6.6s	-	-	135776.3589...	0.149776062...
● ridge	✓ 3 days ago	2.4min	184905.9682...	0.196171131...	120614.8227...	0.170662199...
● baseline_median	✓ 3 days ago	7.7s	-	-	415936.8490...	0.452976398...

เทียบ 4 โมเดล (VALIDATION SET)
- TUNING ALPHA 5 ค่า → เลือก 1.0

Ridge WIN : MAE 120,615 (ยิ่งต่ำยิ่งดี)

ผลโมเดล

```
PS C:\Users\User\UsedCarPrice-mlops> docker run --rm used-car-demo python -m src.explain_model
model_export/model.pkl = version 4, registry champion = version 4
```

=== Top 20 coefficients (เรียงตาม |coef|) ===

#	feature	coef	ผลต่อราคา	หน่วย
1	brand = Tata	-0.503	-39.5%	เมื่อ เป็นหมวดนี้
2	brand = Land	0.485	+62.4%	เมื่อ เป็นหมวดนี้
3	year	0.424	+52.9%	ต่อ +1 SD (3.8)
4	brand = Chevrolet	-0.399	-32.9%	เมื่อ เป็นหมวดนี้
5	brand = Mercedes-Benz	0.373	+45.2%	เมื่อ เป็นหมวดนี้
6	brand = Datsun	-0.356	-29.9%	เมื่อ เป็นหมวดนี้
7	brand = BMW	0.312	+36.6%	เมื่อ เป็นหมวดนี้
8	brand = Volvo	0.300	+35.0%	เมื่อ เป็นหมวดนี้
9	brand = Fiat	-0.256	-22.6%	เมื่อ เป็นหมวดนี้
10	max_power	0.248	+28.2%	ต่อ +1 SD (31.1)
11	brand = Force	-0.214	-19.3%	เมื่อ เป็นหมวดนี้
12	brand = Audi	0.212	+23.6%	เมื่อ เป็นหมวดนี้
13	brand = Jaguar	0.203	+22.5%	เมื่อ เป็นหมวดนี้
14	brand = Daewoo	0.192	+21.1%	เมื่อ เป็นหมวดนี้
15	fuel = Diesel	0.191	+21.1%	เมื่อ เป็นหมวดนี้
16	brand = Toyota	0.173	+18.9%	เมื่อ เป็นหมวดนี้
17	brand = Ashok	-0.157	-14.5%	เมื่อ เป็นหมวดนี้
18	brand = Mahindra	-0.155	-14.4%	เมื่อ เป็นหมวดนี้
19	brand = Isuzu	-0.152	-14.1%	เมื่อ เป็นหมวดนี้
20	fuel = CNG	-0.142	-13.2%	เมื่อ เป็นหมวดนี้

=== ความสำคัญรายคอลัมน์ต้นฉบับ ===

1	brand	หมวดหมู่	0.988
2	year	ตัวเลข	0.424
3	fuel	หมวดหมู่	0.333
4	max_power	ตัวเลข	0.248
5	owner	หมวดหมู่	0.147
6	engine	ตัวเลข	0.098
7	transmission	หมวดหมู่	0.082
8	seller_type	หมวดหมู่	0.071
9	seats	ตัวเลข	0.046
10	km_driven	ตัวเลข	0.021
11	torque	ตัวเลข	0.007
12	mileage	ตัวเลข	0.004

BRAND = Tata -39.5%
BRAND = Land +62.4%

Feature Importance

BRAND	0.988
YEAR	0.424
FUEL	0.333
MAX_POWER	0.248

(ยิ่งสูง = มีผลต่อราคามาก)

MLflow Model Registry

Experiments > used-car-price >

Models

Sort: Created Columns Group by

Model name	Status	Created	Logged from	Source run	Registered models
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	ridge_retrain	used-car-price v4
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	ridge_retrain	used-car-price v3
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	hist_gbm	-
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	random_forest	-
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	ridge	used-car-price v2
model	Ready	1 day ago	C:\Users\User\UsedCarPrice-ml	baseline_median	-
model	Ready	3 days ago	C:\Users\User\UsedCarPrice-ml	hist_gbm	-
model	Ready	3 days ago	C:\Users\User\UsedCarPrice-ml	random_forest	-
model	Ready	3 days ago	C:\Users\User\UsedCarPrice-ml	ridge	used-car-price v1
model	Ready	3 days ago	C:\Users\User\UsedCarPrice-ml	baseline_median	-

Status = Ready
Model version = v1 – v4

- ทุกการทดลองถูกบันทึกลง MLFLOW อัตโนมัติ
- โมเดลอื่นที่ไม่ได้ขึ้นทะเบียนแสดงเป็น "-"

Model Registry

Experiments > used-car-price >

Runs ⓘ

metrics.rmse < 1 and params.model = "tree" ⓘ All time ▾ State: Active ▾ Dataset

				Metrics			
☐	Run Name	Created ↕	Duration	test_mae	test_mape	val_mae	val_mape
☐	ridge_retrain	✓ หลัง RETRAIN	6.7s	177921.5655...	0.192225471...	-	-
☐	ridge_retrain	✓ 1 day ago	5.6s	177921.5655...	0.192225471...	-	-
☐	hist_gbm	✓ 1 day ago	5.3s	-	-	129150.3227...	0.143912674...
☐	random_forest	✓ 1 day ago	5.1s	-	-	135776.3589...	0.149776062...
☐	ridge	✓ ก่อน RETRAIN	21.5s	184905.9682...	0.196171131...	120614.8227...	0.170662199...
☐	baseline_median	✓ 1 day ago	7.1s	-	-	415936.8490...	0.452976398...
☐	hist_gbm	✓ 3 days ago	4.6s	-	-	129150.3227...	0.143912674...
☐	random_forest	✓ 3 days ago	6.6s	-	-	135776.3589...	0.149776062...
☐	ridge	✓ 3 days ago	2.4min	184905.9682...	0.196171131...	120614.8227...	0.170662199...
☐	baseline_median	✓ 3 days ago	7.7s	-	-	415936.8490...	0.452976398...

MAE 184,906 → 177,922

- RETRAIN: รวมข้อมูล VALIDATION เข้า TRAIN
- ROLLBACK ย้อนเวอร์ชันได้ทันที



Live Demo

API มี 3 ส่วนหลัก: ทำนายราคา / ตรวจสอบสถานะระบบ / ดูสถิติการใช้งาน

Used Car Price API 0.1.0 OAS 3.1

[/openapi.json](#)

ระบบทำนายราคารถมือสอง — หน้าใช้งานภาษาไทยอยู่ที่ [/](#) · เอกสารนี้ใช้งานออฟไลน์ได้

ทำนายราคา ส่งสเปกกรมมือสอง แล้วได้ราคาที่เหมาะสม (บาทและรูปี)

POST `/predict` ทำนายราคารถมือสอง

เฝ้าระวังระบบ ตรวจว่า API พร้อมใช้งานและดูสถิติการใช้งาน

GET `/health` ตรวจสอบสถานะระบบ (Health Check)

GET `/metrics` ดูสถิติการใช้งาน (Metrics)

Schemas

Car > Expand all `object`

Prediction > Expand all `object`

กรอกสเปกรถ → ได้ราคาทำนายเป็นบาททันที

ทำนายราคารถมือสอง

กรอกสเปกรถ แล้วกด “ทำนายราคา” — ช่องที่ไม่ทราบเว้นว่างได้

ชื่อยี่ห้อและรุ่น

Maruti Swift Dzire VDI

ปีรถ (1980–2026)

2014

เลขไมล์ (กม.)

145500

เชื้อเพลิง

ดีเซล (Diesel)

เกียร์

ธรรมดา (Manual)

ประเภทผู้ขาย

บุคคลทั่วไป (Individual)

เจ้าของ

มือหนึ่ง

อัตราสิ้นเปลือง

23.4

กม./ลิตร

ขนาดเครื่องยนต์

1248

CC

กำลังสูงสุด

74

bhp

แรงบิด

190

Nm

จำนวนที่นั่ง (2–14)

5

ทำนายราคา

ใช้ข้อมูลตัวอย่าง

ราคาที่ทำนาย

\$170,741

≈ 437,799 บาท · โมเดล v4

ประเมินจากข้อมูลตลาดอินเดีย แปลงเป็นบาทที่ 1 INR = 0.39 THB
เพื่อการศึกษา

ใส่จำนวนที่นั่ง 100 → ระบบปฏิเสธ ไม่ทนายราคามั่ว

ทำนายราคารถมือสอง

กรอกสเปกรถ แล้วกด "ทำนายราคา" — ช่องที่ไม่ทราบเว้นว่างได้

ชื่อยี่ห้อและรุ่น

Maruti Swift Dzire VDI

ปีรถ (1980–2026)

2014

เลขไมล์ (กม.)

145500

เชื้อเพลิง

ดีเซล (Diesel)

เกียร์

ธรรมดา (Manual)

ประเภทผู้ขาย

บุคคลทั่วไป (Individual)

เจ้าของ

มือหนึ่ง

อัตราสิ้นเปลือง

23.4

กม./ลิตร

ขนาดเครื่องยนต์

1248

CC

กำลังสูงสุด

74

bhp

แรงบิด

190

Nm

จำนวนที่นั่ง (2–14)

100

ทำนายราคา

ใช้ข้อมูลตัวอย่าง

ข้อมูลไม่ถูกต้อง กรุณาแก้ไข

- จำนวนที่นั่ง (seats) ต้องไม่เกิน 14

Docker + SLO

Docker: healthy
PORTS: 8000

SLO :

P95 $\leq$ 250 MS $\rightarrow$ ได้ 95.6 MS

THROUGHPUT $\geq$ 50 REQ/S $\rightarrow$ ได้ 211.6 REQ/S

ERROR RATE $\leq$ 1% $\rightarrow$ ได้ 0%

```
PS C:\Users\User\UsedCarPrice-mlops> docker ps --filter name=used-car-api
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS
2f4e690fcdfb   used-car-api   "python -m uvicorn s..." 11 seconds ago Up 11 seconds (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
```



Monitoring

```
saved docs/model_explanation.md
PS C:\Users\User\UsedCarPrice-mlops> docker run --rm used-car-demo python -m src.monitor_drift
saved reports/drift_report.html
reference = data/processed/train.csv (5100 แถว), current = data/processed/test.csv (1018 แถว)
drifted columns = 11/12 (share 91.7%, เกณฑ์ 50%)
DATA DRIFT DETECTED: 11/12 คอลัมน์ (91.7%) drift เกินเกณฑ์ 50% -> การกระจายของข้อมูลเปลี่ยนไป ควรพิจารณาเทรนโมเดลใหม่
```

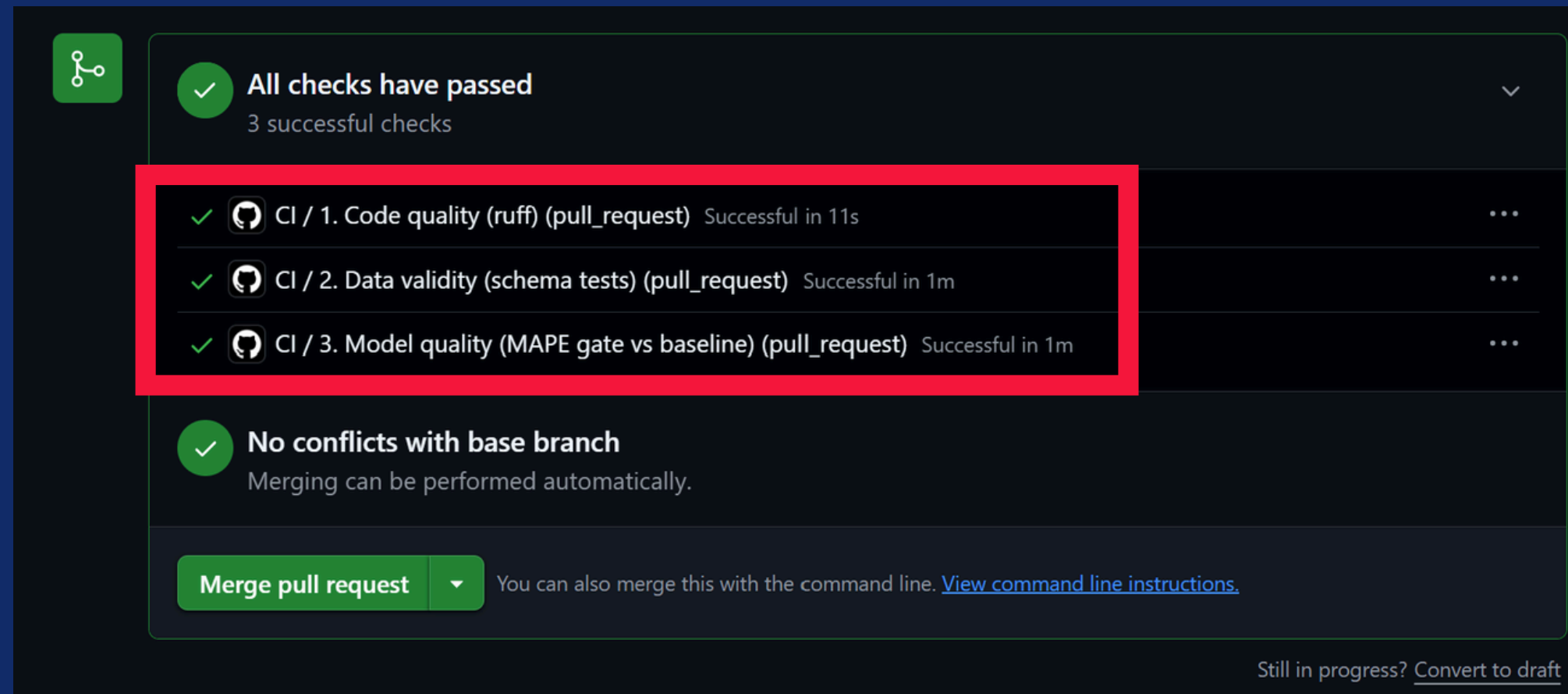
DATA DRIFT
11 จาก 12 คอลัมน์เปลี่ยน (91.7%)
เกินเกณฑ์ 50% → แจ้งเตือน

CONCEPT DRIFT
ความคลาดเคลื่อนเพิ่ม 1.53 เท่า เกินเกณฑ์
1.3 เท่า → แจ้งเตือน

```
PS C:\Users\User\UsedCarPrice-mlops> docker run --rm used-car-demo python -m src.monitor_concept_drift
champion = version 4
ref MAE (2018, จาก MLflow) = 146,433
current MAE (2019-2020, 410 แถว) = 224,618
ratio (current/ref) = 1.53
CONCEPT DRIFT DETECTED: error เพิ่มขึ้น 53% (เกินเกณฑ์ 1.3x) -> ความสัมพันธ์ feature-ราคาเปลี่ยนไป ควรพิจารณาเทรนโมเดลใหม่
```

→ ควรเทรนโมเดลใหม่

CI/CD

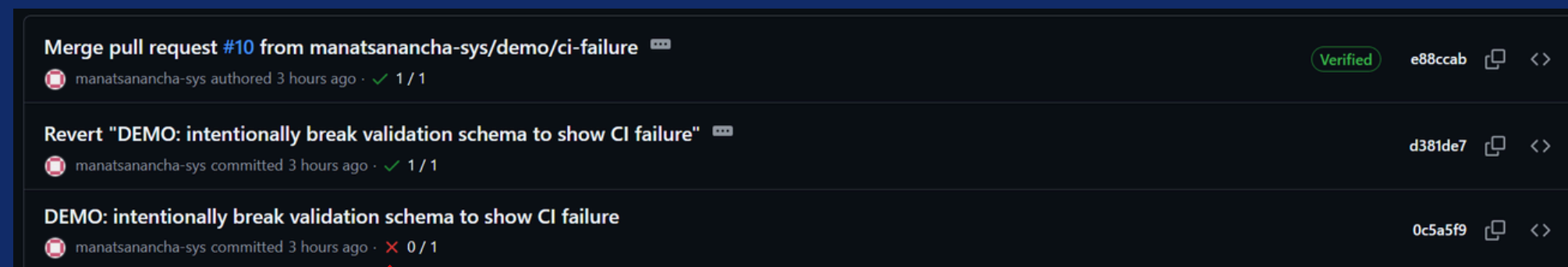


The screenshot shows a GitHub Actions workflow status for a pull request. At the top, a green checkmark indicates "All checks have passed" with "3 successful checks". Below this, a red box highlights three specific checks, all of which are successful:

- ✓ CI / 1. Code quality (ruff) (pull_request) Successful in 11s
- ✓ CI / 2. Data validity (schema tests) (pull_request) Successful in 1m
- ✓ CI / 3. Model quality (MAPE gate vs baseline) (pull_request) Successful in 1m

Below the highlighted checks, another green checkmark indicates "No conflicts with base branch" with the note "Merging can be performed automatically." At the bottom, there is a green button labeled "Merge pull request" and a link to "View command line instructions." The status at the bottom right says "Still in progress? Convert to draft".

- CI ตรวจสอบอัตโนมัติ 3 ด้านทุกครั้งที่ได้แก้โค้ด
CODE QUALITY / DATA VALIDITY / MODEL QUALITY
- ทดสอบใส่บักตั้งใจ → CI จับได้
- แก้กลับ → CI ผ่าน



The screenshot shows the commit history of a pull request. The first two commits are successful:

- Merge pull request #10 from manatsanancha-sys/demo/ci-failure (Verified, e88ccab)
- Revert "DEMO: intentionally break validation schema to show CI failure" (d381de7)

The third commit is highlighted with a red arrow pointing to it from below:

- DEMO: intentionally break validation schema to show CI failure (0c5a5f9)

The status for the third commit shows a red 'X' and "0 / 1", indicating a failed CI run.

ข้อจำกัด

1. ข้อมูลเป็นตลาดอินเดีย (ราคา रुपีย)

แปลงเป็นบาทด้วยอัตราคงที่ $1 \text{ INR} = 0.39 \text{ THB}$ จึงไม่ใช่ราคาตลาดไทยจริง

2. เจอ DRIFT ชัดเจน

DATA DRIFT 91.7% ของคอลัมน์ และ CONCEPT DRIFT(MAE เพิ่ม 1.53 เท่า) โมเดลจึงเสื่อมลงเมื่อเจอข้อมูลปีใหม่

3. ความแม่นยำยังมีขีดจำกัด

MAPE 17.1% และ MAE 177ม922 रुपีย ผ่านเกณฑ์ 20% แบบเฉียดๆ (19.2%)

4. โมเดลเป็น RIDGE (เชิงเส้น)

จับความสัมพันธ์ซับซ้อนได้จำกัด

5. ข้อมูลมีขนาดไม่มาก

มีการ TRAIN ประมาณ 5,100 แถวและไม่มีปัจจัยอย่างสภาพรถหรืออุบัติเหตุ

6. ต้นทุนเป็นเชิงคุณภาพ

ไม่ใช่ตัวเลขจริง เหมาะกับโปรเจกต์ต้นแบบ

สรุป

ปัญหา

การตั้งราคารถมือสองด้วยกฎเขียนเองทำได้ยาก เพราะมีปัจจัยหลายอย่างพร้อมกัน

ทางแก้

ใช้ RIDGE REGRESSION ทำนายราคา แม่นกว่า BASELINE ประมาณ 2.6 เท่า (MAPE 45.3% → 17.1%)

ระบบครบวงจร

ตรวจสอบข้อมูล (422), MLFLOW REGISTRY + ROLLBACK, FASTAPI ใน DOCKER, CI/CD 3 ด้าน

ผ่านเกณฑ์

SLO: P95=95.6MS, THROUGHPUT = 211.6 REQ/S, ERROR = 0%

MONITORING

ตรวจจับ DRIFT ได้และ RETRAIN แล้ว MAE ลดจาก 184,906 เหลือ 177,922

ต่อยอด

ใช้ข้อมูลตลาดไทย, ลองโมเดลที่ซับซ้อนขึ้น, ทำ RETRAIN อัตโนมัติ

Q&A



Thank you